

面向 AI Agent 轨迹的可配置语义剖析

AgentSight 研究原型

2026 年 7 月 3 日

摘要

AI agent 的执行历史已经不再只是一次 prompt 或一棵 trace tree。一个真实轨迹可能同时包含用户意图、LLM 响应、tool call、API 调用、GUI 动作、浏览器动作、进程、文件、网络、计划步骤、subagent 以及 benchmark 给出的成功、失败、安全、重复和冗余标签。现有 agent observability 工具通常把这些对象固定成 prompt、session 或 span 层级；传统 profiler 则固定成函数或进程层级。这两种做法都会把 profiling 问题过早绑定到某一种边界上，使同一段轨迹很难按任务、阶段、工具、动作、质量标签或人工分组递归折叠。

本文提出一种更小的语义剖析模型：系统只保留两个核心抽象，**operation** 和 **operation stack**。**operation** 是带字段和权重的观测，prompt、session、tool call、process、syscall、GUI action 和 plan step 都只是 operation 的不同形态或字段。**operation stack** 是用户从字段中选择的递归投影，mapping 和 tagging 在 stack 构造前派生字段，而不是创建第三种 profiler 对象。我们在 Rust agentpprof 中实现该模型，支持 `--view`、`--stack`、`--op-map`、`--op-map-file` 和可复现实验用的 `--profile-spec`，并把 15 个公开标注 agent 轨迹数据源投影为同一类 operation JSONL。

评估使用 WebLINX、WebShop、API-Bank、Agent-Trek、SWE-agent、Mind2Web、AndroidControl、GUI-Odyssey、ToolBench、tau-bench、AgentRewardBench、SATraj-OS、OSWorld-Human、AgentNet 和 ScaleCUA 的轻量采样，不同步完整测试集或图片归档。核心 14 数据集包含 42,590 个 operations，补充 ScaleCUA 后为 47,590 个 operations。同一批 operations 可以从 9 个 dataset stacks 递归展开到 57 个 phase stacks、226 个 tool/semantic stacks、455 个 action stacks，加入固定 session 后膨胀到 3,757 个 stacks。在 OSWorld-Human 中，单动作轨迹与人工 grouped-action 边界的保守匹配达到 0.627 F1 且 precision 为 1.0；在 AgentNet 中，1,000 个 Ubuntu 人工桌面任务的 16,741 个 PyAutoGUI operations 全部携带 step correctness 和 redundancy 字段；在 AgentRewardBench 中，action-derived repetition signal 对 expert looping label 的 V-measure 从单步错误标签的 0.011 提升到 0.378。R293 进一步把 AgentNet 的同一 operation-stack 查询写成 profile spec：默认 spec 复现 608 个 stacks，命令行只覆盖 `--stack` 后在同一 16,741 个 operations 上得到 83 个更粗粒度 stacks。这些结果支持一个机制性 claim：agent 轨迹可以被统一为 operations，并通过可配置 mappings 和 stack specs 递归折叠为多个有用的 analysis views。当前结果仍不支持完整无监督边界发现或人类用户效用 claim。

1 问题

AI agent 轨迹的调试问题正在从单次调用回放变成跨任务剖析问题。开发者不只想知道某个 span 什么时候开始和结束，也想知道哪类任务导致最多重复点击，哪些 GUI 轨迹

反复输入，哪个工具调用族对应失败，哪些安全攻击类型集中在某些操作阶段，或者一个 prompt 序列能否被折叠成 task、phase、action 和人工 subtask 等多个深度。如果 profiler 把 session、prompt 或 tool call 写死成核心层级，这些问题会被迫使用同一个边界。如果 profiler 只保留进程或函数栈，上层语义和 benchmark oracle 又会丢失。

本文的出发点是把 agent trace 中所有可剖析对象都降到同一层。一个 prompt 可以是 operation，一个 tool call 可以是 operation，一串 prompt 也可以通过 mapping 被派生为同一个 intent 或 task operation field。同样，GUI action、API call、browser action、process event、syscall、安全标签、step correctness 和人工 grouped-action id 都不是新的 profiler 抽象，而是 operation fields。Profiler 的核心任务不是决定唯一正确边界，而是让用户和实验可以在同一批 operations 上选择 stack 字段、派生 mappings，并得到不同深度的递归折叠。

这个问题有两个系统挑战。第一，抽象必须足够小，否则 prompt、session、tool call、plan、subagent 和 process 会各自形成独立 pipeline。第二，抽象又必须足够可配置，否则 flamegraph 只会成为固定视图，不能回答边界、失败、安全和质量诊断问题。本文的目标不是证明某个单一 flamegraph 最好，而是证明 operation 和 operation stack 足以覆盖多类真实标注 agent 轨迹，并能产生 flamegraph 以外的质量、边界和转移分析。

2 设计

2.1 两个核心抽象

operation 是唯一的样本级对象。每个 operation 是一个带权重的字段集合，例如 `op=prompt`、`op=llm`、`op=tool`、`tool=browser`、`action=click`、`phase=navigate`、`status=failure` 或 `step_correct=incorrect`。这些字段可以来自本地 Codex/Claude session，也可以来自外部 benchmark 的标注轨迹。实现上，外部轨迹使用每行一个 JSON object 的 operation JSONL，并把原始任务文本、截图和长对话默认排除在提交 artifact 之外。

operation stack 是从 operation fields 中选择出的有序 frame 列表。例如同一批 operations 可以用 `project,dataset` 查看数据来源，也可以用 `project,dataset,task,phase,op,tool,action,status` 查看动作深度，或者加入 `human_group`、`step_correct`、`safety` 和 `looping` 做诊断视图。Stack 不是 trace tree，也不是 session hierarchy。它是一个查询结果，用户可以按问题改变深度和字段。

2.2 Mapping 和 tagging

Mapping 和 tagging 是派生 operation fields 的一等机制。它们在 stack 构造前运行，读取 operation 的 `key=value` 文本，并写入新的字段。例如 desktop action 中的 `type` 和 `key` 应先被映射到 `phase=input`，再考虑通用 web action verb；API/tool traces 应先进入 tool/API phase family，再

处理 `search` 或 `create` 这类词。这一顺序是 R285、R289、R290 和 R291 暴露出的关键设计约束。如果通用 `action rule` 抢在 `operation-family rule` 前面运行，`desktop`、`web` 和 `API` 轨迹会被错误合并。

Mapping 不引入第三个抽象。它只是在 `operation` 上写字段，使同一个 `stack specification` 可以跨数据集复用。当前 Rust CLI 支持 `inline --op-map FIELD:LABEL=REGEX` 和文件形式 `--op-map-file`。`script/operation_map_infer.py` 可以从已标注 `operations` 中生成同样格式的规则文件，R281–R285 使用这些规则测试 `held-out` 和 `leave-dataset-out` 泛化。`--profile-spec` 把 `operation files`、`mapping files`、`view`、`stack` 和输出路径打包成 JSON 配置，便于 reviewer 复现实验；命令行参数仍可覆盖 `spec` 默认值。它不改变模型本身，只是把已有 `operation` 和 `operation-stack` 查询显式记录下来。

2.3 Views 和非火焰图分析

`--view` 决定采样和权重，例如 `operation count`、`token`、`file`、`network` 或 `time`。`--stack` 决定递归折叠形状。`Folded stack` 和 `SVG flamegraph` 是其中一种输出，但不是唯一输出。我们同时生成 `JSON profile`、`stack tree`、`top-field table`、`parent-child transition`、`depth sweep`、`boundary F1`、`V-measure`、`coverage report`、`grouped-action report` 和 `HTML quality report`。这些视图共享同一 `operation` 输入和 `stack` 语法，因此 `failure`、`safety`、`looping`、`redundancy` 和 `human group` 可以被当作普通字段评分，而不是另建 `failure profiler` 或 `safety profiler`。

3 实现

`agentpprof` 是当前被测实现。它是 Rust CLI，读取本地 `Codex/Claude` 历史或外部 `operation JSONL`，并输出 `pprof-compatible profile`、`folded stacks`、`SVG` 和 `JSON`。核心命令形态如下：

```
agentpprof --operation-file ops.jsonl --view operations
--stack project,dataset,task,phase,op,tool,
action,status --op-map-file operation-map.txt
-o out.folded
```

外部数据集转换器位于 `script/agent_trace_datasets.py`。转换器只下载或流式读取所需公开文件的采样前缀。对于包含截图、长 `prompt` 或完整对话的数据集，默认保存 `redacted rows` 和 `operation JSONL`，不提交原始数据。质量评估脚本位于 `script/operation_stack_quality.py`，它复用 `agentpprof` 的 `operation mapping contract`，并计算字段覆盖率、`oracle V-measure` 和序列相邻边界 `F1`。`Stack` 结构分析脚本位于 `script/operation_stack_analysis.py`，用于生成 `flamegraph` 之外的 `tree`、`top-kind` 和 `transition` 报告。

这个实现刻意保留简单接口。它不像 `jq` 那样完整表达 `JSON` 查询语言，但它已经把 `profiler` 的关键自由度暴露为 `flags` 和可提交配置：数据源由 `--operation-file` 选择，权重由 `--view` 选择，字段派生由 `--op-map` 选择，递归边界由 `--stack` 选择，复现实验由 `--profile-spec` 固化。因此 `prompt`、`session`、`toolcall`、`process`、`syscall`、`plan` 和 `subagent` 都可以在同一套 `mechanism` 中出现，而不是成为互不兼容的 `object model`。

4 实验方法

评估遵循三个原则。第一，只使用公开的、别人已经标注好的 `agent` 轨迹或交互轨迹。我们使用 `Hugging Face Dataset Viewer`、`HF repo JSONL`、公开 `GitHub JSON` 和公开 `benchmark artifacts` 进行轻量采样，不同步完整测试集，也不提交原始外部样本。第二，每个 `claim` 都必须对应可执行 `oracle`。对于 `phase/action` 结构使用 `V-measure` 和相邻边界 `F1`；对于 `grouped-action` 使用人工 `group id`；对于 `looping`、`safety`、`step correctness` 和 `redundancy` 使用数据集原生标签；对于递归深度使用同一批 `operations` 下 `unique stack` 和 `compression` 变化。第三，负结果保留在论文中。例如 `AgentNet` 的 `task status` 很弱地预测 `per-step correctness`，`repeat signal` 也很弱地预测 `step redundancy`。这说明 `profiler` 能承载这些诊断字段，但简单 `mapping` 不能替代专门质量模型。

5 结果

5.1 RQ1: 两个抽象是否覆盖多类轨迹？

核心 14 数据集产生 42,590 个 `operations` 和 1,497 个 `unique stacks`，`compression ratio` 为 28.45。加入 R292 `ScaleCUA` 补充样本后，总量为 47,590 个 `operations` 和 1,611 个 `unique stacks`，`compression ratio` 为 29.541。这些 `operations` 覆盖 `web`、`mobile`、`desktop`、`software`、`API`、`tool-agent-user dialogue`、`expert failure labels`、`safety labels`、`human grouped-action labels` 和 `desktop step-quality labels`。所有这些标签都以 `operation fields` 形式进入同一 `pipeline`。没有任何数据集需要新增 `prompt`、`session`、`GUI`、`safety` 或 `quality-specific profiler object`。

这支持 C1 的机制性版本：`agentpprof` 可以把异构 `agent` 轨迹统一为 `operations`，并用用户指定 `stack` 折叠。它不支持所有 `agent` 数据都容易转换。适合该方法的轨迹通常具备四个特征：有有序 `step` 或 `turn`，有稳定 `session/task id`，有动作或阶段标签，有 `outcome`、`quality`、`group` 或 `safety` 等较高层 `oracle`。只有静态截图 `grounding`、缺少顺序、缺少 `action label`、强依赖大图片归档或 `gated access` 的数据源更适合作为后续扩展，而不是当前主证据。

5.2 RQ2: 递归 stack 是否比固定边界有价值？

R286 在同一批 13,265 个 `operations` 上扫过 8 个 `stack depths`。只保留 `dataset` 时有 9 个 `stacks`；加入 `task` 后为 11 个；`phase depth` 为 57 个；`tool/semantic depth` 为 226 个；`action depth` 为 455 个；加入固定 `session` 后膨胀到 3,757 个。相对于 `dataset depth`，固定 `session` 的 `unique stacks` 增加 417.4 倍。这个结果直接反驳把 `session` 或 `prompt` 作为默认 `profiling` 边界的设计。`Session` 是有用 `drilldown field`，但如果默认写死在 `stack` 里，它会把可聚合的行为切碎。

R277 在较早 3,797-`operation set` 上给出同样信号。`Flat/mapped stack` 为 103 个 `unique stacks`，而固定 `demo/session stack` 为 1,083 个 `unique stacks`。这说明固定边界会带来约 10.5 倍碎片化，而 `mapped operation stack` 可以保留聚合同时增加语义深度。因此论文中的 `novelty` 不是“画 `flamegraph`”，而是把 `agent profiling` 的边界选择变成 `operation-stack query`。

R293 把这个 `query` 自由度变成可复现实验配置。同一个 `AgentNet spec` 引用 R291 的 `operation JSONL` 和 `op-map` 文件，默认产生 608 个 `diagnostic stacks`；命令行

表 1: 公开标注轨迹数据源和当前用途。ScaleCUA 是 R292 补充点, 主结论主要依赖 R279–R291。

数据源	原生标注	转换方式	主要用途
WebLINX, Mind2Web, Agent-Trek, WebShop, API-Bank, Tool-Bench, tau-bench	web action、demo/task、reward 或 action history	HF Dataset Viewer 或 HF repo shard	web navigation 和 shopping trajectories 的跨源 smoke。
SWE-agent	API/tool call、solution path、多轮 user/assistant/tool messages、outcome	HF rows 或 repo JSONL	tool/API/dialogue phases 和 user-agent-tool operation 形态。
AndroidControl, GUI-Odyssey, AgentRewardBench	issue trajectory、command action、success target	HF Dataset Viewer	software-agent command 轨迹。
SATraj-OS	mobile/cross-app step action 与 instruction	public/HF samples	移动 GUI action depth 和跨 app scale。
OSWorld-Human	expert success、side-effect、looping、optimality	annotation rows 加 cleaned BrowserGym steps	failure、looping、side-effect 的非 flame-graph 诊断。
AgentNet	desktop action、success、safety、reward、attack type	HF safety config	桌面安全和 attack-type operation fields。
ScaleCUA	human single-action 和 grouped-action 轨迹	GitHub JSONL files	人工 grouped-action 边界 oracle。
	human desktop PyAutoGUI、task outcome、step correctness、redundancy	HF repo JSONL streaming	最大 human desktop step-quality oracle。
	GUI navigation action、previous-operation context、gold action	HF annotation JSONL streaming	补充多步 GUI history-depth smoke。

只覆盖 `--stack` 和输出路径后, 样本数保持 16,741 不变, unique stacks 降到 83。这说明 stack 深度是可审计、可覆盖的查询参数, 而不是写死在 prompt/session 或某个可视化中的层级。

5.3 RQ3: mapping 是否能泛化?

R281 从 13,265 个 labeled operations 中生成 10 条 mapping rules, 复现手写 mapping 的 folded 输出。R282 使用 dataset-stratified held-out split, 在 9,275 个 train operations 上生成规则, 并在 3,990 个 held-out operations 上评估。Mapped stack 产生 209 个 held-out stacks, compression ratio 为 19.091; 无 mapping baseline 为 284 个 stacks, compression ratio 为 14.049。Task/dataset V-measure 从 0.837 提升到 0.853。Phase/action boundary F1 为 0.777, precision 为 1.0, 说明当前 mapping 是保守的。它不会制造 false positive phase changes, 但会合并一些 fine-grained action changes。

R283–R285 进一步做 leave-dataset-out。在修正 API/tool phase precedence 后, R285 对 9 个 held-out datasets 中的 6 个减少 unique stacks, 0 个出现 negative stack reduction, weighted stack reduction 为每 1,000 operations 1162.005。这说明 mapping 不是单个数据集的 ad hoc pretty printer。它仍然是 deterministic taxonomy-seeded mapping, 不是无监督边界发现。当前可支持的 claim 是 configurable mapping 能在 held-out 和 leave-dataset-out 设置下改善语义聚合; 完整 model-backed 或 unsupervised boundary detector 仍是后续工作。

5.4 RQ4: 能否恢复真实人工边界?

OSWorld-Human 是当前最强的 grouped-boundary oracle。R290 转换全部 369 个 human desktop tasks, 得到 6,010 个 single-action operations。转换器把 grouped-action 序列展平后与 single-action 序列对齐, 只有 exact-aligned tasks 才写入 human_group、group_pattern 和 group_position 字段。最终 320 个 tasks、4,011 个 operations 和 2,075 个 session-local groups 进入 grouped-boundary scoring; 31 个 content-mismatch tasks 和 18 个 length-mismatch tasks 保留用于 action profiling, 但不被当作 gold group oracle。

在 OSWorld-Human 上, 普通 action-depth stack 有 482 个 unique stacks, grouped-depth stack 降到 106 个。Phase/action boundary F1 为 0.638, precision 为 1.0。更

重要的是, 保守的 group_pattern 对人工 human_group boundary 的 F1 为 0.627, precision 为 1.0, recall 为 0.456。这不是完美边界 detector, 但它给出有意义结论: operation stack 可以在同一条单动作序列上选择 action-depth 或 group-depth, 并以人工 grouped-action 标签作为 oracle 验证。Prompt 或 session 没有参与这个抽象。

5.5 RQ5: 能否做失败、安全和质量诊断?

AgentRewardBench、SATraj-OS 和 AgentNet 说明 operation stack 不局限于 flamegraph 热点。AgentRewardBench 的 729 个 expert-reviewed web-agent operations 全部携带 success、side-effect、looping 和 optimality 字段。Action-derived repeat_signal 对 expert looping label 的 V-measure 为 0.378, 而单步 step_error 对 looping 的 V-measure 只有 0.011。这说明 sequence-derived operation fields 可以捕捉一部分真实 failure mode, 也说明简单错误标签不能替代序列诊断。

SATraj-OS 的 4,285 个 desktop computer-use operations 全部携带 safety、attack type、status 和 reward-related fields。其中 622 个 operations 标为 unsafe, attack type 覆盖 account、induced-text、popup、OS 和 unknown-file。这些标签与 phase/action 关系较弱, 例如 attack-type/action V-measure 只有 0.093。这个负结果是有价值的: 安全攻击类别不是 action taxonomy 的简单函数, 因此应该作为独立 operation field 被折叠和过滤, 而不是强行塞进 phase。

AgentNet 是当前最大的 human desktop step-quality oracle。R291 采样 1,000 个 Ubuntu tasks, 得到 16,741 个 PyAutoGUI operations。所有 operations 都有 task outcome、alignment score、efficiency score、difficulty、step correctness 和 redundancy 字段。Step correctness 覆盖 13,844 correct、874 incorrect 和 2,023 unknown; step redundancy 覆盖 9,334 necessary、733 redundant 和 6,674 unknown。Task status 对 step correctness 的 V-measure 只有 0.015, repeat signal 对 step redundancy 的 V-measure 只有 0.010。这说明 profiler 的价值不是用一个粗标签预测所有质量问题, 而是把质量 oracle 保留为可折叠、可 drilldown、可评分的 operation fields。

6 哪些数据集最适合

从当前 15 个方案看, 最适合支撑论文主 claim 的不是“最大”数据集, 而是同时具备顺序、动作、边界或质量 oracle 的

表 2: 当前最强结果。数值均来自 tracked artifacts 下的 JSON/HTML reports。

结果块	数据规模	关键数值	支持的结论
R286 recursive depth	13,265 operations	9 dataset stacks, 57 phase stacks, 226 tool/semantic stacks, 455 action stacks, 3,757 fixed-session stacks	Stack 深度是 query, 固定 session 是 drilldown 而不是核心边界。
R282/R285 mapping validation	3,990 held-out operations; 9 leave-out datasets	Held-out compression 19.091 vs no-map 14.049; leave-out 6/9 positive, 0 negative	Label-derived mappings 能泛化到 unseen sessions/datasets, 但仍不是无监督 detector。
R290 OSWorld-Human	369 tasks, 6,010 operations; 4,011 exact grouped-oracle operations	grouped stack 106 vs action stack 482; human-group boundary F1 0.627, precision 1.0	同一单动作序列可折叠到人工 grouped-action 深度。
R291 AgentNet	1,000 tasks, 16,741 operations	step correctness/redundancy 100% field coverage; phase/action F1 0.715, precision 1.0	质量标签是 ordinary operation fields, 可被折叠、评分和诊断。
R293 profile spec	同一 16,741 AgentNet operations	spec 复现 608 stacks; CLI stack override 产生 83 stacks	实验配置可复现且可覆盖, stack 不是固定层级。
R288/R289 diagnostics	729 AgentRewardBench ops; 4,285 SATraj ops	repeat-signal/looping V-measure 0.378 vs step-error 0.011; 622 unsafe SATraj ops	Profiler 支持 failure/safety views, 且负结果能揭示粗标签不足。

数据集。第一类是 OSWorld-Human, 因为它同时给 single-action 和 grouped-action, 是验证递归边界的最直接 oracle。第二类是 AgentNet, 因为它规模大、人工桌面任务多, 并带 per-step correctness/redundancy。第三类是 AgentRewardBench 和 SATraj-OS, 因为它们把 profiler 从 action taxonomy 推到 failure、looping、安全和 side-effect diagnostics。第四类是 GUI-Odyssey/tau-bench/ToolBench 这组跨域补充, 它们覆盖移动 GUI、user-agent-tool dialogue 和 API/tool traces, 证明 operation abstraction 不局限于桌面。

ScaleCUA 是有用补充, 但不是当前主证据。R292 流式采样 5,000 个 Ubuntu navigation rows, 覆盖 131 个 sessions, 最大 step 为 48。该子集的 action taxonomy 主要是 click 和 terminate, 所以 phase/action 指标达到 1.0 并不表示强泛化。它更适合证明 multi-step GUI history context 可以被投影为 history_state 和 history_depth fields, 而不是证明复杂边界 detector。

7 相关工作

Profiling 和 flamegraphs. 传统 flamegraph 把函数调用栈折叠为热点视图。本文复用 folded-stack 表示, 但 stack frame 来自 operation fields, 而不是语言运行时函数调用。因此同一批 operations 可以被折叠为 task、phase、tool、action、quality 或 safety 视图。这与 CPU profiler 的固定调用栈不同, 也与只画一次 agent span tree 的 tracing 工具不同。

Agent tracing 和 LLM observability. LangSmith、Langfuse、Phoenix、AgentOps 和 OpenTelemetry GenAI 等系统记录 LLM call、tool call、latency、token 和 trace tree。这些工具适合单次 workflow inspection。本文关注的是跨轨迹的 semantic profiling: 把异构标注轨迹或本地 agent 历史统一为 operations, 并允许用户选择 stack 深度和 mappings。

Computer-use 和 tool-use benchmarks. WebLINX、Mind2Web、GUI-Odyssey、OSWorld-Human、AgentNet、SATraj-OS、AgentRewardBench、ToolBench 和 tau-bench 为本文提供真实标注轨迹。本文不提出新 benchmark。贡献是把这些 benchmark 的动作、质量、安全和人工 group 标签统一为 operation fields, 并用同一个 profiler 机制比较它们适合回答哪些 profiling 问题。

8 局限

当前结果仍不是 OSDI 或 NeurIPS 最终接收级完整证据。第一, boundary detection 主要是 deterministic mapping 和 label-derived rule inference。R282/R285 证明它们有 held-out 和 leave-dataset-out 泛化, 但不证明无监督或 LLM-backed boundary detector 已经解决。第二, 用户效用还没有完成。我们证明了 profiler 能产生更结构化的 views 和可执行 oracle, 但没有证明开发者使用这些 views 会更快、更准确。第三, 外部数据集采样为了避免同步完整测试集而保持轻量, 有些数据源只覆盖前缀、子配置或 redacted rows。第四, 非文本/图像密集数据集仍需要更好的 streaming 和 artifact policy, 才能在不保存图片归档的前提下做更大规模评估。第五, paper 当前证据支持 “operation/operation-stack 是一致且有价值的 profiling abstraction”, 不支持 “所有 agent 行为都能自动恢复真实意图边界”。

9 结论

本文把 agent semantic profiling 收敛为两个核心抽象: operation 和 operation stack。Prompt、session、toolcall、process、syscall、plan、subagent、GUI action、安全标签和质量标签都只是 operation shapes 或 fields。Mapping 和 tagging 负责派生 fields, --view 和 --stack 负责选择权重和递归折叠形状, --profile-spec 只把这些选择变成可复现实验配置。在 15 个公开标注轨迹数据源上的结果显示, 这个模型能统一 web、mobile、desktop、software、API、dialogue、failure、安全、human group 和 step-quality 轨迹, 并产生 flamegraph、stack tree、transition、quality 和 boundary reports。最强证据来自 OSWorld-Human 的人工 grouped-action 边界、AgentNet 的大规模 human desktop step-quality 标签、AgentRewardBench/SATraj 的 failure/safety diagnostics, 以及 R286 的递归深度消融。下一步不是继续无差别增加数据集, 而是把 mapping backend、boundary detector 和用户任务实验推进到能支持更强 claim 的程度。

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] OpenTelemetry GenAI semantic conventions. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>.
- [3] McGill-NLP WebLINX. <https://huggingface.co/datasets/McGill-NLP/WebLINX>.
- [4] OpenGVLab GUI-Odyssey. <https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey>.
- [5] WukLab OSWorld-Human. <https://github.com/WukLab/osworld-human>.
- [6] xlangai AgentNet. <https://huggingface.co/datasets/xlangai/AgentNet>.
- [7] McGill-NLP AgentRewardBench. <https://huggingface.co/datasets/McGill-NLP/agent-reward-bench>.
- [8] AI45Research SATraj-OS. <https://huggingface.co/datasets/AI45Research/SATraj-OS>.
- [9] AgentSuite tau-bench trajectories. <https://huggingface.co/datasets/AgentSuite/tau-bench-trajectories>.
- [10] OpenGVLab ScaleCUA-Data. <https://huggingface.co/datasets/OpenGVLab/ScaleCUA-Data>.